Architectural Analysis and Integration Strategy: Persistent Mastery in Stateless Systems

Executive Summary

The rapid evolution of autonomous coding agents has created a fundamental tension between two architectural philosophies: the "Stateful" agent, which accumulates context and preferences over time in a local environment, and the "Stateless" agent, which relies on a deterministic, version-controlled definition of reality for every session. This report provides an exhaustive technical analysis of the everything-claude-code repository, a paradigmatic example of a stateful, continuous-learning architecture, and evaluates its compatibility with the strict, stateless governance model of the Stela platform.

The analysis reveals that everything-claude-code achieves "Memory Persistence" through a system of lifecycle hooks (session-start, session-end, evaluate-session) that read and write to a local, hidden directory (~/.claude/) outside the repository's version control.[1] This approach creates a "Hidden State" that directly violates Stela's core constitutional invariant: "Context is not memory; it is a file".[3] In Stela's architecture, defined by TRUTH.md and AGENTS.md, any state that is not visible in the repository dump essentially does not exist. Therefore, the "out-of-the-box" implementation of everything-claude-code is incompatible with Stela's governance model because it introduces untracked drift and subverts the Operator's authority by allowing the Contractor (AI) to modify its own instructions without an explicit commit.

However, the functional capability of "Continuous Learning"—the ability to crystallize repeatable patterns from work logs into reusable instructions—is highly desirable. This report proposes a "Persistent Mastery" integration plan that adapts the *logic* of pattern extraction while rejecting the *mechanism* of hidden state. By establishing a new canonical directory (docs/library/skills/) and implementing a "Crystallization" workflow managed by the Integrator, Stela can achieve persistent mastery. This approach converts implicit "Instincts" into explicit, version-controlled "Skills" that are ingested via the standard ops/bin/dump mechanism, ensuring that the system learns and evolves while remaining fully stateless, auditable, and compliant with TRUTH.md.

1. Introduction: The Divergence of Stateful and Stateless Architectures

The fundamental challenge in modern AI agent orchestration lies in the management of

context. As agents perform complex tasks—ranging from refactoring codebases to managing infrastructure—they generate valuable "meta-knowledge": which commands failed, which architectural patterns were preferred by the human operator, and which sequences of actions led to success. The preservation of this meta-knowledge distinguishes a "learning" system from a repetitive one.

In the broader landscape of AI engineering, two distinct philosophies have emerged to handle this preservation. The first, exemplified by everything-claude-code, is the **Stateful** approach. This philosophy treats the AI agent as a persistent entity—a "digital employee"—that maintains a distinct identity and memory bank on the local machine.[1] This agent "wakes up" with memories of past projects, preferences, and "instincts" that are injected into its context window automatically. The state is local to the machine, personal to the user, and persists across different repositories.

The second philosophy, exemplified by Stela, is the **Stateless** approach. This philosophy treats the AI agent as a transient "Contractor".[3] The agent has no identity or memory beyond the files explicitly provided to it at the start of a session. "Rehydration" is the process of loading the repository state into the agent's context window. If a piece of information is not in the file dump, the agent does not know it. This approach prioritizes reproducibility, auditability, and team synchronization over individual agent convenience.

The conflict between these two models is not merely technical; it is jurisdictional. In a Stateful model, the agent accumulates authority through its hidden preferences. In a Stateless model, authority is strictly reserved for the "Constitution" (TRUTH.md) and the "Contract" (TASK.md). Integrating the capabilities of the former into the constraints of the latter requires a sophisticated re-engineering of how "Memory" is defined, stored, and accessed.

2. Technical Analysis of Target Architecture: everything-claude-code

The everything-claude-code repository represents a maximalist configuration for the Claude Code CLI, designed to transform the tool from a passive interface into an active, learning partner. Its architecture is built upon three pillars: **Memory Persistence Hooks**, **Continuous Learning Logic**, and a **Hierarchical Artifact System**.

2.1 The Mechanism of Memory Persistence

The core technical innovation of everything-claude-code is the use of lifecycle hooks to intercept the agent's startup and shutdown phases. These hooks allow the execution of arbitrary Node.js scripts that manipulate the agent's environment.[4]

2.1.1 The Session Lifecycle

The repository defines a specific lifecycle for every coding session, governed by the

hooks/hooks.json configuration file. This file maps specific events—such as SessionStart, SessionEnd, and Stop—to executable scripts.

- **Initialization (session-start.js):** When the user initializes a session, the SessionStart hook triggers node scripts/hooks/session-start.js.[4] This script performs a crucial function: it scans the local environment for persistent context. Unlike Stela, which dumps the current repository, this script looks into ~/.claude/sessions/ (a directory in the user's home folder) to find the most recent "Session Log." It retrieves the state of the previous interaction, including open tasks, unsolved errors, and user preferences, and injects this text directly into the system prompt or the initial user message. This creates the illusion of continuity; the agent "remembers" what it was doing yesterday, even if the computer was restarted.
- **Termination (session-end.js):** When the user exits the tool, the SessionEnd hook triggers node scripts/hooks/session-end.js.[4] This script serializes the active context. It captures the conversation history, the status of any checklists, and the "Instincts" generated during the session (discussed below). It saves this data to a timestamped file (e.g., YYYY-MM-DD-topic.tmp) in the local storage directory.[1] This ensures that no insight is lost when the volatile RAM of the context window is cleared.
- **Implications:**
  The reliance on ~/.claude/ creates a **Hidden State**. This data is not part of the project repository. If a developer switches machines, or if a different team member works on the same codebase, this "Memory" is lost or inconsistent. The state is attached to the *developer's machine*, not the *code*.

2.1.2 Continuous Learning and Pattern Extraction

Beyond simple logging, everything-claude-code implements a primitive form of metacognition through the evaluate-session.js script.[4] This script runs post-session and performs a "Reflective" pass over the transcript.

- **Pattern Recognition Logic:**
  The script analyzes the interaction for specific signals. It looks for moments where the user corrected the agent (e.g., "No, use the functional style, not classes"), moments where the agent successfully resolved a recurring error, or explicit commands to "remember this."
- **Crystallization:** When a pattern is identified, it is formatted into a structured data object known as an "Instinct." If the confidence score for an instinct is high (e.g., >0.8), or if the pattern has been observed multiple times, the system may upgrade it to a full "Skill".[5]
- **Storage:**
  These learned patterns are saved to ~/.claude/skills/learned/. This directory effectively becomes the agent's "Subconscious," filled with heuristics that guide its behavior in future sessions without explicit prompting.

2.2 The Artifact Taxonomy

The target repository distinguishes between two primary types of knowledge artifacts: **Skills** and **Instincts**.

2.2.1 Skills (SKILL.md)

Skills are the "heavyweight" artifacts of the system. They are structured Markdown documents designed to teach the agent a complex workflow or a specific domain capability.[6]

Schema Analysis:

A typical SKILL.md file follows a rigorous structure to ensure the LLM can interpret it correctly:

- **Frontmatter:** YAML metadata defining the skill's name, description, and required tools (e.g., grep, bash).
- **When to Use:** A specific section defining the triggers for the skill (e.g., "When writing Go code," "When refactoring legacy PHP").[7]
- **Core Principles:** The philosophical "Constitution" of the skill (e.g., "Simplicity over cleverness").
- **Execution Steps:** A numbered list of actions the agent must take.
- **Examples:** Concrete "Good vs. Bad" code snippets to ground the generation.

This structure is highly compatible with LLM reasoning. By providing explicit "When to Use" triggers, the system allows the agent to self-select the appropriate skill from a library, rather than requiring the user to paste instructions manually.

2.2.2 Instincts (YAML/JSON)

Instincts are the "lightweight" artifacts, introduced in the "Continuous Learning v2" update.[1] They represent granular behavioral nudges rather than full workflows.

Schema Analysis:

YAML

```yaml
instincts:
  - id: prefer-functional-style
    trigger: "when writing new functions"
    action: "Use functional patterns over classes"
    confidence: 0.8
    domain: code-style
    observations: 8
```

These artifacts are designed for aggregation. The system uses an /evolve command [8] to

cluster related instincts. For example, if the agent collects five instincts related to "Error Handling," the /evolve command might merge them into a new error-handling-skill.md.

2.3 Environmental Coupling

A critical aspect of the everything-claude-code architecture is its tight coupling with the local development environment.

- **Package Manager Detection:** The system includes logic (setup-package-manager.js) to automatically detect whether the project uses npm, pnpm, yarn, or bun.[1] It checks environment variables, lock files, and global configs.
- **Token Optimization:** The architecture includes background processes to "slim" the system prompt and optimize context usage.[1] This is necessary because carrying a persistent memory load can quickly consume the limited context window of the LLM.

This coupling highlights the "Stateful" philosophy: the agent adapts to the environment it finds itself in, rather than enforcing a rigid environment defined by the repository.

3. The Stela Constitution: A Forensic Review

To understand the friction between the target architecture and Stela, we must first rigorously define the Stela "Constitution" as established in TRUTH.md and AGENTS.md. Stela is not merely a repository; it is a governance system designed to prevent "Drift"—the gradual entropy of software projects caused by undocumented decisions and hidden states.

3.1 The "Context is a File" Doctrine

The central dogma of Stela is defined in docs/CONTEXT.md: **"Context is not memory; it is a file."**.[3]

In a stateless system, the agent (Contractor) is presumed to have zero knowledge at the start of a session. It effectively dies at the end of a task and is reborn at the start of the next. To function, it must be "rehydrated" with the current state of the world. This state is captured exclusively by the ops/bin/dump tool.

- **Implication:** If a file exists on the disk (e.g., in ~/.claude/) but is not included in the ops/bin/dump output, it does not exist in the Stela universe.
- **Security:** This ensures that no "Secret Knowledge" drives the system. Every decision the agent makes can be audited by reviewing the dump that was provided to it. There are no "hidden variables" in the equation.

3.2 Staffing Protocol and Jurisdiction

AGENTS.md establishes a strict hierarchy of authority.[3]

- **Operator (Human):** Owns final decisions and performs all commits.

- **Integrator (Lead AI):** Maintains governance and detects drift.
- **Contractor (Guest AI):** Executes specific tasks.

The "Contractor" is explicitly forbidden from inventing repository structures or defining its own scope. It must adhere to the **Behavioral Logic Standard** of "Absolute Literalism".[3] This clashes directly with the everything-claude-code concept of an agent that "learns" and "evolves" instincts. In Stela, an agent that "evolves" on its own is an agent that has gone rogue. Any evolution must be codified in TRUTH.md by the Operator.

3.3 The State Capture Mechanism: ops/bin/dump

The ops/bin/dump tool is the technical enforcement of Stela's philosophy.[3]

- **Scope:** It captures the "Platform" (ops/, docs/, tools/) and "Project" (projects/) directories.
- **Mechanism:** It uses git ls-files to identify tracked content. This means it intentionally ignores untracked files (like local logs) unless they are explicitly added to the dump manifest.
- **Constraint:** The tool is rooted at ./ (the repo root) and does not have visibility into parent directories or the user's home directory. This is a deliberate design choice to ensure hermeticity.

4. Fit Gap Analysis

Having analyzed both systems, we can now map the conflicts and opportunities. The core conflict is between **Hidden State** (Target) and **Visible State** (Stela).

4.1 Conflict 1: The Location of Memory (~/.claude vs. Repo Root)

Dimension	everything-claude-code	Stela Requirement	Impact
Storage Location	~/.claude/ (Home Dir)	Repo Root (./)	**Critical Failure**
Visibility	Hidden from Git	Version Controlled	**Critical Failure**
Access	session-start.js Hook	ops/bin/dump	**Incompatible**

The reliance on ~/.claude/ is a "Critical Failure" for Stela. Stela requires that all configuration and memory be portable. If the Operator clones the repo to a new machine, the "Memory"

must come with it. everything-claude-code breaks this invariant.

Resolution: "Memory" artifacts must be moved from ~/.claude/ to a canonical directory within the repository, such as docs/library/skills/.

4.2 Conflict 2: The Logic of Drift (Auto-Write vs. Operator Commit)

Dimension	everything-claude-code	Stela Requirement	Impact
Skill Creation	Automated (evaluate-session.js)	Manual (SoP.md update)	**Governance Risk**
Authority	Agent Self-Evolution	Operator Approval	**Governance Risk**
Verification	Confidence Score	Repo Gates	**Gap**

The evaluate-session.js script automatically writes new skills based on its own analysis. In Stela, this constitutes "Drift." An agent cannot change the system's rules (skills) without Operator approval.

Resolution: The *logic* of pattern extraction is valid, but the *execution* must change. Instead of writing files directly, the agent must propose a "Skill Draft" (e.g., in storage/memory/) which the Operator (or Integrator) reviews and commits to the docs/library/skills/ directory.

4.3 Conflict 3: Context Injection (Implicit vs. Explicit)

Dimension	everything-claude-code	Stela Requirement	Impact
Injection	Implicit (Hidden Hook)	Explicit (OPEN Prompt)	**Transparency Risk**
Auditability	Low (Magic behavior)	High (All context in file)	**Transparency Risk**

In the target architecture, hooks silently inject prompts. An Operator looking at the terminal might not know *why* the agent is behaving a certain way. Stela mandates that all context be

visible in the OPEN prompt artifact.

Resolution: We must disable the hidden hooks (session-start.js) and instead update the ops/bin/open script to explicitly read from the docs/library/skills/ directory and append relevant skills to the OPEN prompt.

4.4 The "Continuous Learning" Opportunity

Despite these conflicts, the *functional* capability of learning from the past is highly compatible with Stela's "Equilibrium Maintenance" directive. AGENTS.md states: "A task is considered complete only when the system state reaches equilibrium and SoP.md is updated."

Currently, Stela uses SoP.md as a history ledger ("What happened"). everything-claude-code uses SKILL.md as an executable ledger ("How to do it"). By integrating SKILL.md artifacts, we can upgrade Stela's memory from a passive log to an active instruction set, provided we wrap it in the correct governance.

5. The "Persistent Mastery" Integration Architecture

This section defines the new architecture for Stela: **Persistent Mastery**. This architecture adapts the "Continuous Learning" capabilities of everything-claude-code but implements them through Stela's "Stateless" mechanisms.

5.1 Concept: Crystallization vs. Persistence

We replace the concept of "Persistence" (keeping state alive) with "Crystallization" (hardening state into files).

- **Phase 1: Liquid Memory (Work Log).** During a session, the agent logs insights, errors, and fixes into the TASK.md "Work Log" section. This corresponds to the session-end.js logs in the target repo, but it lives inside the repo's active contract file.
- **Phase 2: Extraction (The Memento).** At the end of a session, instead of running a background script, the Integrator runs a specific **Memento** (a standardized prompt) called M-LEARN-01. This prompts the agent to review the Work Log and identify reusable patterns.
- **Phase 3: Crystallization (The Skill).** If a pattern is identified, the agent drafts a SKILL.md file in a temporary holding area (storage/memory/).
- **Phase 4: Canonization (The Commit).** The Operator reviews the draft. If valid, it is moved to docs/library/skills/ and committed. Once committed, it becomes part of the "Context" for all future sessions.

5.2 Directory Structure Updates

We must establish new "Coordinates" in TRUTH.md to house these artifacts. The target repo's ~/.claude/ structure is mapped to Stela's repo structure as follows:

Target Path (~/.claude/)	Stela Canonical Path	Governance
skills/learned/	docs/library/skills/	**Canon** (Read-only, SSOT)
sessions/	storage/archive/logs/	**Museum** (History only)
instincts.yaml	storage/memory/instincts.yaml	**Scratch** (Temporary)

5.3 The "Skill" Artifact Standard

We will adopt the SKILL.md format from everything-claude-code but enforce Stela-specific headers to ensure it aligns with TRUTH.md.

Template Location: docs/library/skills/TEMPLATE.md

Schema:

SKILL:[Name]

Status: CANON

Maintainer: Integrator

Source:

1. Trigger (When to Load)

- **Primary:**
- **Secondary:** [File pattern, e.g., "*.module"]

2. Invariants (Constitution)

- [Hard Constraint 1]
- [Hard Constraint 2]

3. Protocol (Execution)

1. Step 1...
2. Step 2...

4. Verification (Gate)

- [How to prove success]

5.4 Adapting ops/bin/dump for Skills

The existing ops/bin/dump tool captures the platform state. To support Persistent Mastery, it must be updated to ingest the docs/library/skills/ directory.

Technical Change:

The ops/bin/dump script must be modified to accept a --skills flag or to automatically include the skills directory when --scope=platform is selected.

- **Logic:** It should scan docs/library/skills/ for .md files.
- **Filtering:** It should interpret the "Trigger" section of the skills. Ideally, ops/bin/open (the prompt generator) should perform a lightweight grep of the user's intent against the "Trigger" sections of the skills library. If a match is found, that specific skill file is added to the required_files list passed to dump.

5.5 The M-LEARN-01 Protocol

Instead of evaluate-session.js, we define a **Memento**—a codified interaction pattern stored in docs/library/MEMENTOS.md.

Protocol Definition:

> **M-LEARN-01: Pattern Extraction**
>
> **Trigger:** Executed by Integrator during TASK.md closeout.
>
> **Prompt:** "Review the 'Work Log' of the current TASK. Identify any recurring errors, successful debugging sequences, or architectural preferences that deviated from the initial plan. If a pattern appears valuable for future sessions, format it as a SKILL draft and save it to storage/memory/."

This replaces the "black box" Node.js script with a transparent, Operator-supervised linguistic process.

6. Implementation Roadmap

This checklist provides the step-by-step actions for the Operator to upgrade Stela with Persistent Mastery capabilities.

Phase 1: Infrastructure Preparation (The Container)

1. **Establish Coordinates:**
    - Create directory: mkdir -p docs/library/skills
    - Create directory: mkdir -p storage/memory
    - Add .gitkeep to both.
2. **Define Template:**
    - Create docs/library/skills/TEMPLATE.md using the schema defined in Section 5.3.
3. **Update Constitution (TRUTH.md):**
    - Edit TRUTH.md Section 3 (Structure) to include docs/library/skills/ as the canonical home for learned behaviors.
    - Add Invariant: "Skills are crystallized memory. They override generic training but are subservient to TRUTH.md."
4. **Update Context Manifest:**
    - Edit ops/lib/manifests/CONTEXT.md.
    - Add docs/library/skills/ to the "Supporting Surfaces" list.
    - *Verification:* Run tools/context_lint.sh to confirm it detects the new directory.

Phase 2: Logic Integration (The Brain)

5. **Codify the Memento:**
    - Edit docs/library/MEMENTOS.md.
    - Add the M-LEARN-01 protocol defined in Section 5.5.
6. **Update ops/bin/open:**
    - Modify ops/bin/open to scan docs/library/skills/ and list available skills in the `` section of the generated prompt.
    - *Logic:* Use ls docs/library/skills/*.md to append a list of "Available Skills" to the context.
7. **Update TASK.md Template:**
    - Edit the 3) CLOSEOUT section of TASK.md.
    - Add a checkbox: [ ] Run M-LEARN-01: Check for crystallized skills?

Phase 3: Pilot and Verification

8. **Pilot Execution:**
    - Run a standard session.
    - At closeout, manually invoke M-LEARN-01.
    - Draft a dummy skill (e.g., git-discipline.md) based on the session.
    - Save it to storage/memory/.
9. **Promotion:**
    - Operator reviews storage/memory/git-discipline.md.
    - Operator moves file to docs/library/skills/git-discipline.md.
    - Operator commits changes.
10. **Verification Loop:**
    - Start a new session using ops/bin/open.
    - Verify that git-discipline.md is visible in the context or listed as an available pointer.

- Verify tools/lint_truth.sh passes.

7. Conclusion

The everything-claude-code repository demonstrates a sophisticated "Stateful" architecture that prioritizes agent autonomy and local persistence. While its specific mechanisms—hidden hooks, local storage, and automated writing—are incompatible with Stela's "Stateless" and strict governance model, the underlying concept of **Persistent Mastery** is essential for high-functioning AI operations.

By implementing the integration plan outlined above, Stela effectively absorbs the "Continuous Learning" capability of the target repository without compromising its constitution. We replace **Hidden State** with **Canon Artifacts** (SKILL.md), and **Automated Drift** with **Managed Crystallization** (M-LEARN-01). This results in a system that learns and evolves, but does so transparently, ensuring that the "Memory" of the agent remains as auditable and version-controlled as the code itself. The agent no longer just works; it masters the craft, session by session, under the strict supervision of the Operator.

Works cited

1. affaan-m/everything-claude-code: Complete Claude Code configuration collection - agents, skills, hooks, commands, rules, MCPs. Battle-tested configs from an Anthropic hackathon winner. - GitHub, accessed January 30, 2026, https://github.com/affaan-m/everything-claude-code
2. PS-Chronicles-First-Semester-2024-2025.pdf - BITS Pilani, accessed January 30, 2026, https://www.bits-pilani.ac.in/wp-content/uploads/PS-Chronicles-First-Semester-2024-2025.pdf
3. dump-full-main-f38a29f56.txt
4. hooks.json - affaan-m/everything-claude-code · GitHub, accessed January 30, 2026, https://github.com/affaan-m/everything-claude-code/blob/main/hooks/hooks.json
5. Everything Claude Code Part 1 - 概要と全24コマンド詳細 | アイドリ | AI-Driven Lab - note, accessed January 30, 2026, https://note.com/ai_driven/n/n52eb1b38e146
6. everything-claude-code/CONTRIBUTING.md at main - GitHub, accessed January 30, 2026, https://github.com/affaan-m/everything-claude-code/blob/main/CONTRIBUTING.md
7. golang-patterns by affaan-m/everything-claude-code - Skills, accessed January 30, 2026, https://skills.sh/affaan-m/everything-claude-code/golang-patterns
8. continuous-learning-v2 by affaan-m/everything-claude-code - Skills, accessed January 30, 2026, https://skills.sh/affaan-m/everything-claude-code/continuous-learning-v2